

CS50

Introduction to Databases

Complete Study Notes

SQLite • PostgreSQL • MySQL

Querying • Relating • Designing • Writing • Viewing • Optimizing • Scaling

Lec 1

Relating

Lec 2

Designing

Lec 3

Writing

Lec 4

Viewing

Lec 5

Optimizing

Table of Contents

Lecture 0	Querying — SELECT, WHERE, LIKE, ORDER BY, Aggregate Functions
Lecture 1	Relating — Subqueries, JOINS, Sets, GROUP BY, HAVING
Lecture 2	Designing — CREATE TABLE, Data Types, Constraints, ALTER, DROP
Lecture 3	Writing — INSERT, DELETE, UPDATE, CSV Import, Triggers, Soft Delete
Lecture 4	Viewing — CREATE VIEW, Temporary Views, CTEs, Security, Triggers on Views
Lecture 5	Optimizing — Indexes, EXPLAIN QUERY PLAN, B-Trees, Partial Index, VACUUM, Transactions, ACID, Concurrency
Lecture 6	Scaling — PostgreSQL, MySQL, Replication, Sharding, Stored Procedures, Access Control, SQL Injection

LEC 0 Lecture 0: Querying

Retrieving data from a database using SELECT

0.1 SQLite Setup

Open a database file with sqlite3 and configure display settings:

```
sqlite3 longlist.db -- open (or create) a database file
.system cls -- clear the terminal (Windows)
.mode columns -- display results in aligned columns
.headers yes -- show column names in output
.tables -- list all tables in the database
.schema -- view CREATE statements for all tables
.schema -- view CREATE statement for a specific table
.timer on -- show execution time after each query
```

0.2 Basic SELECT

The SELECT statement retrieves data from a table:

```
SELECT * FROM "longlist"; -- all columns, all rows
SELECT "title", "author" FROM "longlist"; -- specific columns
SELECT "title", "author" FROM "longlist" LIMIT 10; -- first 10 rows only
```

■ **Note:** Use double quotes for identifiers (table/column names) and single quotes for string values. This is SQLite best practice.

0.3 WHERE Clause — Filtering Rows

Filter rows using conditions with WHERE:

```
-- Equality and inequality
SELECT "title", "author" FROM "longlist" WHERE "year" = 2023;
SELECT "title", "format" FROM "longlist" WHERE "format" != 'hardcover';
SELECT "title", "format" FROM "longlist" WHERE "format" <> 'hardcover';
SELECT "title", "format" FROM "longlist" WHERE NOT "format" = 'hardcover';
```

0.3.1 Logical Operators: AND / OR / NOT

```
-- OR: either condition matches
SELECT "title", "author" FROM "longlist"
WHERE "year" = 2022 OR "year" = 2023;

-- AND + OR combined (use parentheses to control precedence!)
SELECT "title", "format" FROM "longlist"
```

```
WHERE ("year" = 2022 OR "year" = 2023)
AND "format" != 'hardcover';
```

0.3.2 NULL Checks

NULL means no value. You cannot use = or != with NULL — use IS NULL / IS NOT NULL:

```
SELECT "title", "translator" FROM "longlist" WHERE "translator" IS NULL;
SELECT "title", "translator" FROM "longlist" WHERE "translator" IS NOT NULL;
```

0.3.3 LIKE — Pattern Matching

LIKE performs case-insensitive pattern matching. Two wildcards are available:

%	Matches any sequence of zero or more characters
_	Matches exactly one character

```
SELECT "title" FROM "longlist" WHERE "title" LIKE '%love%'; -- contains 'love'
SELECT "title" FROM "longlist" WHERE "title" LIKE 'The%'; -- starts with 'The'
SELECT "title" FROM "longlist" WHERE "title" LIKE 'The%Goes%'; -- 'The' then 'Goes'
SELECT "title" FROM "longlist" WHERE "title" LIKE 'P_re'; -- P, any char, re
SELECT "title" FROM "longlist" WHERE "title" LIKE 'T___'; -- T + exactly 3 chars
```

■ **Important:** LIKE is case-insensitive for pattern matching (e.g. 'pyre' matches 'Pyre'). However, = is case-sensitive (e.g. 'pyre' = 'Pyre' returns no result).

0.3.4 Range Filters: Comparison Operators and BETWEEN

```
-- Using comparison operators
SELECT "title", "year" FROM "longlist"
WHERE "year" >= 2019 AND "year" <= 2022;

-- Using BETWEEN (inclusive on both ends)
SELECT "title", "year" FROM "longlist"
WHERE "year" BETWEEN 2019 AND 2022;

-- Numeric conditions
SELECT "title", "rating" FROM "longlist" WHERE "rating" > 4.0;
SELECT "title", "rating", "votes" FROM "longlist"
WHERE "rating" > 4.0 AND "votes" > 10000;
```

0.4 ORDER BY — Sorting Results

```
-- Sort ascending (default)
SELECT "title", "rating" FROM "longlist" ORDER BY "rating" LIMIT 10;

-- Sort descending
SELECT "title", "rating" FROM "longlist" ORDER BY "rating" DESC LIMIT 10;

-- Sort by multiple columns
SELECT "title", "rating", "votes" FROM "longlist"
```

```
ORDER BY "rating" DESC, "votes" DESC
LIMIT 10;
```

0.5 Aggregate Functions

Aggregate functions compute a single value from multiple rows:

Function	Description	Example
COUNT(*)	Total number of rows	SELECT COUNT(*) FROM "longlist";
COUNT(col)	Rows where col is NOT NULL	SELECT COUNT("translator") FROM "longlist";
AVG(col)	Average value	SELECT AVG("rating") FROM "longlist";
SUM(col)	Total sum	SELECT SUM("votes") FROM "longlist";
MAX(col)	Maximum value	SELECT MAX("title") FROM "longlist";
MIN(col)	Minimum value	SELECT MIN("title") FROM "longlist";
ROUND(val,n)	Round to n decimal places	SELECT ROUND(AVG("rating"),2);

```
SELECT AVG("rating") FROM "longlist";
SELECT ROUND(AVG("rating"), 2) FROM "longlist";
SELECT ROUND(AVG("rating"), 2) AS "average rating" FROM "longlist"; -- alias with AS
SELECT SUM("votes") FROM "longlist";
SELECT COUNT(*) FROM "longlist";
SELECT COUNT("translator") FROM "longlist"; -- NULL rows not counted
SELECT MAX("title"), MIN("title") FROM "longlist";
```

0.6 DISTINCT — Unique Values

```
SELECT DISTINCT "publisher" FROM "longlist"; -- unique publishers
SELECT COUNT(DISTINCT "publisher") FROM "longlist"; -- count of unique publishers
```

LEC 1 Lecture 1: Relating

Subqueries, JOINS, Sets, GROUP BY & HAVING

1.1 Subqueries (Nested Queries)

A subquery is a SELECT statement inside another query. It runs first, and its result feeds the outer query.

```
-- Find books published by MacLehose Press (one-step)
SELECT "title" FROM "books" WHERE "publisher_id" = (
  SELECT "id" FROM "publishers"
  WHERE "publisher" = 'MacLehose Press'
);

-- Average rating of a specific book
SELECT AVG("rating") FROM "ratings" WHERE "book_id" = (
  SELECT "id" FROM "books" WHERE "title" = 'In Memory of Memory'
);

-- Many-to-many: find author of 'Flights'
SELECT "name" FROM "authors"
WHERE "id" = (
  SELECT "author_id" FROM "authored"
  WHERE "book_id" = (
    SELECT "id" FROM "books" WHERE "title" = 'Flights'
  )
);
```

1.1.1 IN Keyword — Multiple Results from Subquery

Use IN when a subquery can return multiple rows:

```
-- Find all books by Fernanda Melchor
SELECT "title" FROM "books" WHERE "id" IN (
  SELECT "book_id" FROM "authored"
  WHERE "author_id" = (
    SELECT "id" FROM "authors" WHERE "name" = 'Fernanda Melchor'
  )
);

-- Result: Paradaís, Hurricane Season
```

1.2 JOINS

A JOIN combines rows from two or more tables based on a related column. The general syntax is:

```
SELECT columns FROM table1
```

```
JOIN table2 ON table1.column = table2.column;
```

1.2.1 Types of JOINS

JOIN Type	Description	Rows Included
INNER JOIN (default: JOIN)	Rows that match in BOTH tables	Only matched rows
LEFT JOIN (LEFT OUTER JOIN)	All rows from left table + matched from right	All left + matched right (NULLs for unmatched right)
RIGHT JOIN (RIGHT OUTER JOIN)	All rows from right table + matched from left	All right + matched left (NULLs for unmatched left)
FULL JOIN (FULL OUTER JOIN)	All rows from both tables	All rows; NULLs where no match
NATURAL JOIN	Automatically joins on columns with the same name	Only matched rows (no ON clause needed)

1.2.2 JOIN Examples with sea_lions.db

```
-- INNER JOIN (default): only matching rows
SELECT * FROM "sea_lions"
JOIN "migrations" ON "migrations"."id" = "sea_lions"."id";

-- LEFT JOIN: all sea_lions, even those without migration data
SELECT * FROM "sea_lions"
LEFT JOIN "migrations" ON "migrations"."id" = "sea_lions"."id";

-- Jolee appears with NULL migration values (no match in migrations table)

-- RIGHT JOIN: all migrations, even those without a named sea lion
SELECT * FROM "sea_lions"
RIGHT JOIN "migrations" ON "migrations"."id" = "sea_lions"."id";

-- IDs 11735, 11736, 11737 appear with NULL sea lion names

-- FULL JOIN: everything from both tables
SELECT * FROM "sea_lions"
FULL JOIN "migrations" ON "migrations"."id" = "sea_lions"."id";

-- NATURAL JOIN: auto-joins on shared column name ("id")
-- No ON clause needed; duplicate column not shown
SELECT * FROM "sea_lions"
NATURAL JOIN "migrations";
```

1.3 Sets — UNION, INTERSECT, EXCEPT

Set operations combine results from two SELECT queries. Both queries must return the same number of columns with compatible types.

```
-- UNION: all names from both tables (duplicates removed)
```

```

SELECT "name" FROM "translators"
UNION
SELECT "name" FROM "authors";

-- UNION with a label column (to distinguish source)
SELECT 'author' AS "profession", "name" FROM "authors"
UNION
SELECT 'translator' AS "profession", "name" FROM "translators";

-- INTERSECT: names that appear in BOTH tables
SELECT "name" FROM "authors"
INTERSECT
SELECT "name" FROM "translators";

-- EXCEPT: names in authors but NOT in translators
SELECT "name" FROM "authors"
EXCEPT
SELECT "name" FROM "translators";

-- INTERSECT with subqueries: books translated by BOTH Hughes and Costa
SELECT "book_id" FROM "translated" WHERE "translator_id" = (
  SELECT "id" FROM "translators" WHERE "name" = 'Sophie Hughes'
)
INTERSECT
SELECT "book_id" FROM "translated" WHERE "translator_id" = (
  SELECT "id" FROM "translators" WHERE "name" = 'Margaret Jull Costa'
);

```

1.4 GROUP BY and HAVING

GROUP BY groups rows sharing the same values in specified columns. Aggregate functions then operate per group. HAVING filters groups (like WHERE, but for aggregated results):

```

-- Average rating per book
SELECT "book_id", ROUND(AVG("rating"), 2) AS "average rating"
FROM "ratings"
GROUP BY "book_id";

-- Only books with average rating above 4.0
SELECT "book_id", ROUND(AVG("rating"), 2) AS "average rating"
FROM "ratings"
GROUP BY "book_id"
HAVING "average rating" > 4.0;

-- Count ratings per book
SELECT "book_id", COUNT("rating") FROM "ratings"
GROUP BY "book_id";

```



```
-- Combined: filter + sort
SELECT "book_id", ROUND(AVG("rating"), 2) AS "average rating"
FROM "ratings"
GROUP BY "book_id"
HAVING "average rating" > 4.0
ORDER BY "average rating" DESC;
```

■ **Note:** *WHERE filters rows before grouping. HAVING filters groups after aggregation. You cannot use WHERE with aggregate functions.*

LEC 2 Lecture 2: Designing

CREATE TABLE, Data Types, Constraints, ALTER TABLE, DROP TABLE

2.1 CREATE TABLE

Define a new table with column names and types:

```
-- Minimal table (no types specified – SQLite allows this but not recommended)
CREATE TABLE "riders" (
  "id",
  "name"
);

-- Proper table with types
CREATE TABLE "riders" (
  "id" INTEGER,
  "name" TEXT
);

CREATE TABLE "stations" (
  "id" INTEGER,
  "name" TEXT,
  "line" TEXT
);

-- Junction table for many-to-many relationship
CREATE TABLE "visits" (
  "rider_id" INTEGER,
  "station_id" INTEGER
);
```

2.2 SQLite Data Types

Type	Description
INTEGER	Whole numbers (1, 2, 3, -10, etc.)
REAL	Floating-point numbers (3.14, 2.5, etc.)
TEXT	Strings / character data
NUMERIC	Dates, times, booleans — stored as integer or real depending on value
BLOB	Binary data, stored as-is

2.3 Constraints

Constraints enforce rules on column data at the database level:

```
CREATE TABLE "collections" (  
  "id" INTEGER,  
  "title" TEXT NOT NULL, -- value required  
  "accession_number" TEXT NOT NULL UNIQUE, -- required + no duplicates  
  "acquired" NUMERIC,  
  PRIMARY KEY("id") -- unique row identifier  
);
```

NOT NULL	Column cannot be left empty (NULL)
UNIQUE	All values in the column must be different
PRIMARY KEY	Uniquely identifies each row; implies NOT NULL + UNIQUE. Usually INTEGER for auto-increment
FOREIGN KEY	Links a column to the PRIMARY KEY of another table — enforces referential integrity
CHECK	Custom condition that must be true (e.g. CHECK(amount != 0))
DEFAULT	Value used when no value is provided on INSERT

2.4 Foreign Keys

A FOREIGN KEY creates a link between two tables. The ON DELETE clause controls what happens when the referenced row is deleted:

```
CREATE TABLE "created" (  
  "artist_id" INTEGER,  
  "collection_id" INTEGER,  
  PRIMARY KEY("artist_id", "collection_id"),  
  FOREIGN KEY("artist_id") REFERENCES "artists"("id") ON DELETE CASCADE,  
  FOREIGN KEY("collection_id") REFERENCES "collections"("id") ON DELETE CASCADE  
);
```

ON DELETE Action	Effect
RESTRICT / NO ACTION	Block the delete — raise an error if referenced rows exist
SET NULL	Set the foreign key column to NULL in child rows
SET DEFAULT	Set the foreign key column to its DEFAULT value
CASCADE	Automatically delete child rows when the parent is deleted

2.5 Reading the Schema from a File

```
-- Write schema in a .sql file, then load it:
```

```
sqlite3 mbta.db
.read schema.sql
.schema -- verify the result
```

2.6 ALTER TABLE

Modify an existing table structure:

```
-- Rename a table
ALTER TABLE "visits" RENAME TO "swipes";

-- Add a new column
ALTER TABLE "swipes" ADD COLUMN "type" TEXT;

-- Rename a column
ALTER TABLE "swipes" RENAME COLUMN "type" TO "swipe_type";

-- Drop a column
ALTER TABLE "swipes" DROP COLUMN "type";
```

2.7 DROP TABLE

```
DROP TABLE "riders";
DROP TABLE "stations";
DROP TABLE "visits";
```

■ **Important:** *DROP TABLE permanently removes the table and all its data. There is no undo in SQLite. Always back up before dropping.*

LEC 3 Lecture 3: Writing

INSERT, DELETE, UPDATE, CSV Import, Triggers, Soft Deletes

3.1 INSERT — Adding Rows

```
-- Insert with explicit id
INSERT INTO "collections" ("id","title","accession_number","acquired")
VALUES (1, 'Profusion of flowers', '56.257', '1956-04-12');

-- Insert without specifying id (auto-assigned if PRIMARY KEY)
INSERT INTO "collections" ("title","accession_number","acquired")
VALUES ('Spring outing', '14.76', '1911-01-08');

-- Insert multiple rows at once
INSERT INTO "collections" ("title", "accession_number", "acquired")
VALUES
('Imaginative Landscape', '56.496', NULL),
('Hariyeh', '06.12849', '1906-01-01');
```

3.2 Importing CSV Data

```
-- Import CSV directly into an existing table (skip header row)
.import --csv --skip 1 mfs.csv collections

-- Import CSV into a temporary table, then copy with SELECT
.import --csv mfs2.csv temp

INSERT INTO "collections" ("title", "accession_number", "acquired")
SELECT "title", "accession_number", "acquired" FROM "temp";
```

3.3 DELETE — Removing Rows

```
-- Delete all rows (dangerous!)
DELETE FROM "collections";

-- Delete specific rows
DELETE FROM "collections" WHERE "title" = 'Spring outing';
DELETE FROM "collections" WHERE "acquired" IS NULL;
DELETE FROM "collections" WHERE "acquired" < '1911-01-01';
```

3.3.1 Foreign Key Constraint on Delete

```
-- This fails if 'Unidentified artist' is referenced in 'created' table:
DELETE FROM "artists" WHERE "name" = 'Unidentified artist';

-- Error: Runtime error: FOREIGN KEY constraint failed (19)
```

```
-- Solution: delete the child rows first
DELETE FROM "created" WHERE "artist_id" = (
  SELECT "id" FROM "artists" WHERE "name" = 'Unidentified artist'
);
-- Now the parent can be deleted safely
DELETE FROM "artists" WHERE "name" = 'Unidentified artist';
```

■ **Note:** If you defined `ON DELETE CASCADE` on the foreign key, SQLite deletes child rows automatically when you delete the parent.

3.4 UPDATE — Modifying Rows

```
-- Generic structure
UPDATE table
SET column1 = value1, column2 = value2, ...
WHERE condition;

-- Update a single row using a subquery
UPDATE "created"
SET "artist_id" = (
  SELECT "id" FROM "artists" WHERE "name" = 'Li Yin'
)
WHERE "collection_id" = (
  SELECT "id" FROM "collections" WHERE "title" = 'Farmers working at dawn'
);

-- Text functions in UPDATE
UPDATE "votes" SET "title" = TRIM("title"); -- remove leading/trailing spaces
UPDATE "votes" SET "title" = UPPER("title"); -- convert to uppercase

-- Pattern-based update using LIKE
UPDATE "votes" SET "title" = 'FARMERS WORKING AT DAWN'
WHERE "title" LIKE 'Fa%';

UPDATE "votes" SET "title" = 'IMAGINATIVE LANDSCAPE'
WHERE "title" LIKE 'Imaginative %';

UPDATE "votes" SET "title" = 'PROFUSION OF FLOWERS'
WHERE "title" LIKE 'Profusion%';
```

3.5 Triggers

A trigger is a set of SQL statements that automatically runs when a specific event occurs on a table.

```
-- General syntax
CREATE TRIGGER name
BEFORE/AFTER INSERT ON table
UPDATE OF column ON table
```

```
DELETE ON table
FOR EACH ROW
BEGIN
.....;
END;
```

3.5.1 BEFORE DELETE Trigger — Log sold items

```
CREATE TRIGGER "sell"
BEFORE DELETE ON "collections"
FOR EACH ROW
BEGIN
INSERT INTO "transactions" ("title", "action")
VALUES (OLD."title", 'sold'); -- OLD refers to the row being deleted
END;

-- When we now delete:
DELETE FROM "collections" WHERE "title" = 'Profusion of flowers';

-- The transactions table automatically gets a new row:
SELECT * FROM "transactions";

-- Result: 1 | Profusion of flowers | sold
```

3.5.2 AFTER INSERT Trigger — Log bought items

```
CREATE TRIGGER "buy"
AFTER INSERT ON "collections"
FOR EACH ROW
BEGIN
INSERT INTO "transactions" ("title", "action")
VALUES (NEW."title", 'bought'); -- NEW refers to the row being inserted
END;

-- When we insert:
INSERT INTO "collections" ("title", "accession_number", "acquired")
VALUES ('Profusion of flowers', '56.247', '1956-04-17');

-- The transactions table automatically gets:
-- 1 | Profusion of flowers | sold
-- 2 | Profusion of flowers | bought
```

■ **Note:** OLD references the row before the event (available in DELETE and UPDATE). NEW references the row after the event (available in INSERT and UPDATE).

3.6 Soft Deletes

Instead of physically deleting rows (which is irreversible), soft delete marks them as deleted using a flag column. This allows recovery but has privacy implications.

```
-- Step 1: Add a 'deleted' flag column with default 0 (not deleted)
ALTER TABLE "collections" ADD COLUMN "deleted" INTEGER DEFAULT 0;

-- Step 2: 'Delete' a row by setting the flag
UPDATE "collections" SET "deleted" = 1
WHERE "title" = 'Farmers working at dawn';

-- Step 3: Query only active (non-deleted) rows
SELECT * FROM "collections" WHERE "deleted" != 1;

-- Result after update:
-- 2 | Farmers working at dawn | ... | 1 <- soft-deleted
-- 3 | Profusion of flowers | ... | 0 <- active
```


LEC 4 Lecture 4: Viewing

CREATE VIEW, Temporary Views, CTEs, Security, Triggers on Views

4.1 Why Views?

A VIEW is a saved SELECT query that you can query like a regular table. It simplifies complex queries, provides a security layer, and keeps your SQL DRY (Don't Repeat Yourself).

```
-- Hard way (every time you want author + book):
SELECT "title" FROM "books"
WHERE "id" IN (
  SELECT "book_id" FROM "authored"
  WHERE "author_id" = (
    SELECT "id" FROM "authors" WHERE "name" = 'Gauz'
  )
);

-- Easy way using a JOIN:
SELECT "name", "title" FROM "authors"
JOIN "authored" ON "authors"."id" = "authored"."author_id"
JOIN "books" ON "books"."id" = "authored"."book_id";
```

4.2 CREATE VIEW

```
-- Create a view
CREATE VIEW "longlist" AS
SELECT "name", "title" FROM "authors"
JOIN "authored" ON "authors"."id" = "authored"."author_id"
JOIN "books" ON "books"."id" = "authored"."book_id";

-- Now use it like a table
SELECT * FROM "longlist";
SELECT "title" FROM "longlist" WHERE "name" = 'Gauz';

-- Result: Standing Heavy
```

■ **Note:** Views are virtual — they don't store data. Each time you query a view, the underlying SELECT runs. Views appear in .schema output with a comment showing their columns.

4.3 Aggregating in Views

```
-- View: average rating per book
CREATE VIEW "average_book_ratings" AS
SELECT "book_id", "title", "year",
```

```

ROUND(AVG("rating"), 2) AS "rating"
FROM "ratings"
JOIN "books" ON "ratings"."book_id" = "books"."id"
GROUP BY "book_id";

-- Then query the view
SELECT * FROM "average_book_ratings";

-- Build on top of the view
SELECT "year", ROUND(AVG("rating"), 2) AS "rating"
FROM "average_book_ratings"
GROUP BY "year";

```

4.4 Temporary Views

Temporary views exist only for the current database session. They are deleted when you disconnect.

```

CREATE TEMPORARY VIEW "average_ratings_by_year" AS
SELECT "year", ROUND(AVG("rating"), 2) AS "rating"
FROM "average_book_ratings"
GROUP BY "year";

```

4.5 CTEs — Common Table Expressions

A CTE is a temporary, named result set defined with WITH. It lives only for the duration of the single query. Unlike views, CTEs are not saved to the database.

```

-- Syntax
WITH name AS (
SELECT ...
), another_name AS (
SELECT ...
)
SELECT ... FROM name;

-- Example: replicate a view inline without creating it
WITH "average_book_ratings" AS (
SELECT "book_id", "title", "year",
ROUND(AVG("rating"), 2) AS "rating"
FROM "ratings"
JOIN "books" ON "ratings"."book_id" = "books"."id"
GROUP BY "book_id"
)
SELECT "year", ROUND(AVG("rating"), 2) AS "rating"
FROM "average_book_ratings"
GROUP BY "year";

```

4.6 Security with Views

Create a view that exposes only safe columns, hiding sensitive data (e.g. passenger names) from analysts:

```
-- Full table with sensitive rider info
SELECT * FROM "rides";

-- Anonymised view – 'rider' column replaced with 'Anonymous'
CREATE VIEW "analysis" AS
SELECT "id", "origin", "destination",
'Anonymous' AS "rider"
FROM "rides";

-- Analysts query the view – they never see real names
SELECT * FROM "analysis";
```

4.7 Views Cannot Be Updated (Usually)

```
-- This will raise an error:
UPDATE "2021" SET "title" = 'The Minor Detail'
WHERE "title" = 'Minor Detail';

-- Error: cannot modify view
```

■ **Important:** Standard views in SQLite are read-only. Use *INSTEAD OF* triggers (below) to simulate write operations on views.

4.8 Soft Delete View + INSTEAD OF Trigger

```
-- Create a view that shows only non-deleted rows
CREATE VIEW "current_collections" AS
SELECT "id", "title", "accession_number", "acquired"
FROM "collections"
WHERE "deleted" = 0;

-- Trigger: deleting from the VIEW does a soft delete on the underlying table
CREATE TRIGGER "delete"
INSTEAD OF DELETE ON "current_collections"
FOR EACH ROW
BEGIN
UPDATE "collections" SET "deleted" = 1
WHERE "id" = OLD."id";
END;

-- Trigger: inserting into view re-activates soft-deleted row (if accession exists)
CREATE TRIGGER "insert_when_exists"
INSTEAD OF INSERT ON "current_collections"
FOR EACH ROW
WHEN NEW."accession_number" IN (
```

```
SELECT "accession_number" FROM "collections"
)
BEGIN
UPDATE "collections" SET "deleted" = 0
WHERE "accession_number" = NEW."accession_number";
END;
```

LEC 5 Lecture 5: Optimizing

Indexes, EXPLAIN QUERY PLAN, B-Trees, VACUUM, Transactions, ACID, Concurrency

5.1 The Performance Problem

Without an index, SQLite must scan every single row in a table to find matches (called a full table scan). This is slow on large tables.

```
sqlite3 movies.db
.timer on -- enable timing output

-- Full table scan – slow on a large table
SELECT * FROM "movies" WHERE "title" = 'Cars';

-- Run Time: real 0.523493 ...
```

5.2 CREATE INDEX

An index is a separate data structure (typically a B-Tree) that allows the database to find rows by column value very quickly — without scanning every row.

```
-- Create an index on the 'title' column
CREATE INDEX "title_index" ON "movies" ("title");

-- Run Time: real 0.360618 ... (one-time cost to build)

-- Now the same query is fast:
SELECT * FROM "movies" WHERE "title" = 'Cars';

-- Run Time: real 0.002164 ...

-- Remove an index
DROP INDEX "title_index";
```

5.3 EXPLAIN QUERY PLAN

Shows how SQLite plans to execute a query — whether it will use an index or fall back to a full scan:

```
-- Without index: SCAN (bad on large tables)
EXPLAIN QUERY PLAN
SELECT * FROM "movies" WHERE "title" = 'Cars';

-- QUERY PLAN: `--SCAN movies

-- After creating index: SEARCH using index (fast)
CREATE INDEX "title_index" ON "movies" ("title");
EXPLAIN QUERY PLAN
SELECT * FROM "movies" WHERE "title" = 'Cars';

-- QUERY PLAN: `--SEARCH movies USING INDEX title_index (title=?)
```

5.4 Multi-Column and Covering Indexes

```
-- Before indexes: nested query is slow (SCAN on stars and people)
SELECT "title" FROM "movies" WHERE "id" IN (
  SELECT "movie_id" FROM "stars" WHERE "person_id" = (
    SELECT "id" FROM "people" WHERE "name" = 'Tom Hanks'
  )
);
-- Run Time: real 0.523493 ...

-- Create indexes on the columns used in conditions
CREATE INDEX "name_index" ON "people" ("name");
CREATE INDEX "person_index" ON "stars" ("person_id");
-- Now uses SEARCH instead of SCAN

-- Covering index: includes ALL columns the query needs from that table
-- No separate table lookup required (fastest possible)
DROP INDEX "person_index";
CREATE INDEX "person_index" ON "stars" ("person_id", "movie_id");
-- EXPLAIN shows: SEARCH stars USING COVERING INDEX person_index
```

■ **Note:** A covering index contains all columns needed by the query, so SQLite never has to look up the actual table row — it gets everything from the index itself.

5.5 B-Tree

SQLite uses a B-Tree (Balanced Tree) as the underlying data structure for indexes. Key properties:

- Sorted: values are stored in sorted order, enabling fast range lookups
- Balanced: every path from root to leaf has the same length — $O(\log n)$ search time
- Each node can have multiple children (not just two like a binary tree)
- Related concept: a linked list is $O(n)$ linear scan; a B-Tree is $O(\log n)$ search

5.6 Partial Index

A partial index indexes only rows that satisfy a WHERE condition. Smaller, faster to build, and only benefits queries that match the condition:

```
-- Create an index only for movies from 2023
CREATE INDEX "recents" ON "movies" ("title")
WHERE "year" = 2023;

-- Uses the index:
EXPLAIN QUERY PLAN
SELECT "title" FROM "movies" WHERE "year" = 2023;
-- QUERY PLAN: `--SCAN movies USING COVERING INDEX recents

-- Does NOT use the index (different year):
EXPLAIN QUERY PLAN
```

```
SELECT "title" FROM "movies" WHERE "year" = 1998;
-- QUERY PLAN: `--SCAN movies
```

5.7 VACUUM — Reclaiming Disk Space

Dropping indexes does not immediately shrink the file — SQLite marks space as free but doesn't release it.

VACUUM rebuilds the database and reclaims disk space:

```
-- Before dropping indexes:
$ du -b movies.db
158949376 movies.db

-- After dropping all indexes (file size unchanged):
DROP INDEX "person_index";
DROP INDEX "name_index";
DROP INDEX "recents";
$ du -b movies.db
158949376 movies.db

-- Run VACUUM to actually reclaim space:
VACUUM;
$ du -b movies.db
100388864 movies.db -- 36% smaller!
```

5.8 Transactions and ACID

A transaction groups multiple SQL statements into a single unit of work. Either all statements succeed (COMMIT) or all are rolled back (ROLLBACK).

```
-- Transfer 10 from Alice (id=1) to Bob (id=2)
BEGIN TRANSACTION;
UPDATE "accounts" SET "balance" = "balance" + 10 WHERE "id" = 2;
UPDATE "accounts" SET "balance" = "balance" - 10 WHERE "id" = 1;
COMMIT;

-- If a CHECK constraint fails mid-transaction, rollback:
BEGIN TRANSACTION;
UPDATE "accounts" SET "balance" = "balance" + 10 WHERE "id" = 2;
UPDATE "accounts" SET "balance" = "balance" - 10 WHERE "id" = 1;
-- Runtime error: CHECK constraint failed: balance
ROLLBACK; -- undo everything – balance stays the same
SELECT * FROM "accounts";
-- Alice: 0, Bob: 30, Charlie: 30 (unchanged)
```

5.8.1 ACID Properties

Property	Meaning
----------	---------

Atomicity	All statements in a transaction succeed together or fail together — no partial updates
Consistency	A transaction brings the database from one valid state to another — all constraints remain satisfied
Isolation	Concurrent transactions don't interfere with each other — each sees a consistent snapshot
Durability	Once committed, changes survive crashes or power failures (written to disk)

5.9 Concurrency and Locks

SQLite uses file-level locking to manage concurrent access. Lock states:

UNLOCKED	No transaction active — anyone can read or write
SHARED	A read transaction is active — multiple readers allowed, no writers
EXCLUSIVE	A write transaction is active — no other reads or writes allowed

```
-- Acquire an exclusive lock (blocks all other connections)
BEGIN EXCLUSIVE TRANSACTION;
SELECT * FROM "accounts";
-- While this is open, another terminal gets:
-- Parse error: database is locked (5)
```


LEC 6 Lecture 6: Scaling

PostgreSQL, MySQL, Replication, Sharding, Stored Procedures, Security

6.1 SQLite vs. Server Databases

SQLite	Embedded database — runs inside your application process. Stores data as a single file. Not designed for high-concurrency multi-user production use.
PostgreSQL	Full client-server database — runs as a separate process/hardware. Communicates over a network. Designed for production, scalable, runs in RAM.
MySQL	Another client-server database. Widely used in web applications. Similar concepts to PostgreSQL but different syntax in places.

6.2 Scaling Strategies

6.2.1 Vertical Scaling

Add more CPU, RAM, or storage to a single server. Simple, but has physical limits.

6.2.2 Horizontal Scaling — Replication

Distribute the load across multiple servers by keeping copies of the database (replicas). Three replication types:

Single-Leader	One leader accepts all writes. Followers replicate from it. Read replicas serve read-only queries. Most common setup.
Multi-Leader	Multiple leaders can accept writes. More complex — potential for write conflicts.
Leaderless	Any node can accept writes. Fully distributed. Most complex to keep consistent.

Read Replica	A follower copy of the database used only for reads. Offloads read traffic from the leader.
Synchronous Replication	Leader waits for follower to confirm write before responding to client. Slower but no data loss.
Asynchronous Replication	Leader responds to client immediately; follower catches up later. Faster but potential for data loss on crash.

6.2.3 Sharding

Split a single large database into smaller pieces (shards) across multiple servers. Each shard holds a logical subset of the data (e.g., users A–M on server 1, N–Z on server 2). Sharding increases write capacity but adds application complexity.

6.3 PostgreSQL

```
-- Connect
psql postgresql://postgres@127.0.0.1:5432/postgres

-- Useful commands
\! cls -- clear screen
\l -- list all databases
\c "mbta" -- connect to (use) a database
\dt -- list tables in current database
\d "cards" -- describe a table's structure
```

6.3.1 PostgreSQL Auto-Increment Types

Type	Range
SMALLSERIAL	1 to 32,767
SERIAL	1 to 2,147,483,647
BIGSERIAL	1 to 9,223,372,036,854,775,807

6.3.2 PostgreSQL Data Types

```
TIMESTAMP(p) -- date + time with optional precision
DATE -- date only
TIME(p) -- time only
INTERVAL(p) -- time span (e.g. '3 days', '2 hours')
NUMERIC(precision, scale) -- exact decimal (e.g. NUMERIC(10,2))

-- Custom ENUM type
CREATE TYPE "swipe_type" AS ENUM ('enter', 'exit', 'deposit');
```

6.4 MySQL

```
-- Connect
mysql -u root -h 127.0.0.1 -P 3306 -p

-- Useful commands
SHOW DATABASES; -- list all databases
\! cls OR system cls -- clear screen
USE `mbta`; -- select a database
SHOW TABLES; -- list tables
DESCRIBE `swipes`; -- show column definitions
```

■ **Note:** MySQL uses backticks (`) for identifiers, while SQLite and PostgreSQL use double quotes (").

6.4.1 MySQL Data Types

Category	Types
Integers	INT AUTO_INCREMENT (equivalent to SERIAL in PostgreSQL)
Strings	CHAR(M), VARCHAR(M), TEXT / TINYTEXT / MEDIUMTEXT / LONGTEXT
Binary	BLOB
Special	ENUM (one value from list), SET (multiple values from list)
Date/Time	DATE, TIME(fsp), DATETIME(fsp), TIMESTAMP(fsp), YEAR
Real Numbers	FLOAT, DOUBLE PRECISION, DECIMAL(p,s) (e.g. DECIMAL(6,2))

6.4.2 MySQL CREATE TABLE Example

```
CREATE TABLE `cards` (
  `id` INT AUTO_INCREMENT,
  PRIMARY KEY(`id`)
);

CREATE TABLE `stations` (
  `id` INT AUTO_INCREMENT,
  `name` VARCHAR(32) NOT NULL UNIQUE,
  `line` ENUM('blue','green','orange','red') NOT NULL,
  PRIMARY KEY(`id`)
);

CREATE TABLE `swipes` (
  `id` INT AUTO_INCREMENT,
  `card_id` INT,
  `station_id` INT,
  `type` ENUM('enter','exit','deposit') NOT NULL,
  `datetime` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  `amount` DECIMAL(5,2) NOT NULL CHECK(`amount` != 0),
  PRIMARY KEY(`id`),
  FOREIGN KEY(`card_id`) REFERENCES `cards`(`id`),
  FOREIGN KEY(`station_id`) REFERENCES `stations`(`id`)
);
```

■ **Note:** In DESCRIBE output, MUL in the Key column means the column is part of a non-unique index (typically from a FOREIGN KEY).

6.4.3 MySQL ALTER TABLE

```
-- Change a column's type/properties
ALTER TABLE `stations`
MODIFY `line` ENUM('blue','green','orange','red','silver') NOT NULL;

-- Add a new column
```

```
ALTER TABLE `collections`  
ADD COLUMN `deleted` TINYINT DEFAULT 0;
```

6.5 Stored Procedures (MySQL)

A stored procedure is a saved, named block of SQL that can be called with parameters. Similar to a function. Use DELIMITER to change the statement terminator when defining:

```
-- Procedure with no parameters  
DELIMITER //  
CREATE PROCEDURE `current_collection` ()  
BEGIN  
SELECT `title`, `accession_number`, `acquired`  
FROM `collections` WHERE `deleted` = 0;  
END//  
DELIMITER ;  
  
-- Call it  
CALL `current_collection`();  
  
-- Procedure with an IN parameter (input only)  
DELIMITER //  
CREATE PROCEDURE `sell`(IN `sold_id` INT)  
BEGIN  
UPDATE `collections` SET `deleted` = 1  
WHERE `id` = `sold_id`;  
  
INSERT INTO `transactions` (`title`, `action`)  
VALUES (  
(SELECT `title` FROM `collections` WHERE `id` = `sold_id`),  
'sold'  
);  
END//  
DELIMITER ;  
  
-- Call with argument  
CALL `sell`(2);
```

6.6 Access Control (MySQL)

```
-- Create a new user  
CREATE USER 'carter' IDENTIFIED BY 'password';  
  
-- Grant specific privileges  
GRANT SELECT ON `rideshare`.`analysis` TO 'carter';  
  
-- Available privileges:  
-- ALL, CREATE, INSERT, SELECT, UPDATE, DELETE
```

■ **Note:** A new user with no grants sees only `information_schema` and `performance_schema` — no application databases. Grant only the minimum privileges needed.

6.7 SQL Injection Attacks

SQL injection occurs when user input is concatenated directly into a SQL query, allowing an attacker to alter the query's logic:

```
-- Vulnerable login check – attacker inputs: ' OR '1'='1
SELECT `id` FROM `users`
WHERE `username` = 'Carter'
AND `password` = 'password' OR '1' = '1'; -- always TRUE!

-- Vulnerable balance check – attacker inputs: 1 UNION SELECT * FROM accounts
SELECT `balance` FROM `accounts`
WHERE `id` = 1 UNION SELECT * FROM `accounts`; -- leaks all accounts!
```

6.7.1 Prevention — Prepared Statements

Prepared statements separate the query structure from the data. User input is always treated as a value, never as SQL code:

```
-- Define the query with ? as a placeholder
PREPARE `balance_check`
FROM 'SELECT * FROM `accounts` WHERE `id` = ?';

-- Execute with a safe value
SET @id = 1;
EXECUTE `balance_check` USING @id;

-- Even malicious input is treated as a literal string, not SQL:
SET @id = '1 UNION SELECT * FROM `accounts`';
EXECUTE `balance_check` USING @id;

-- Returns empty result – no injection possible
```

■ **Important:** Always use prepared statements (or parameterised queries in your programming language) whenever user input is used in SQL queries. Never concatenate raw user input into SQL strings.

Quick Reference — SQL Commands Summary

Core SQL Commands

Command	Purpose	Database
SELECT ... FROM ...	Retrieve data	All
WHERE	Filter rows	All
ORDER BY ... [DESC]	Sort results	All
LIMIT n	Return only n rows	All
AS	Alias a column or table	All
LIKE '%pattern%'	Pattern match (case-insensitive)	All
IS NULL / IS NOT NULL	Check for missing values	All
BETWEEN a AND b	Range filter (inclusive)	All
IN (...)	Match against a list or subquery result	All
DISTINCT	Remove duplicate rows	All
COUNT / AVG / SUM / MAX / MIN	Aggregate functions	All
ROUND(val, n)	Round to n decimals	All
TRIM / UPPER / LOWER	String manipulation	All
GROUP BY	Group rows for aggregation	All
HAVING	Filter aggregated groups	All
JOIN / LEFT JOIN / RIGHT JOIN / FULL JOIN	Combine tables	All
NATURAL JOIN	Auto-join on shared column names	SQLite
UNION / INTERSECT / EXCEPT	Set operations	All
CREATE TABLE	Define a new table	All
ALTER TABLE	Modify table structure	All
DROP TABLE	Remove a table	All
INSERT INTO	Add rows	All
UPDATE ... SET	Modify rows	All
DELETE FROM	Remove rows	All
CREATE VIEW / DROP VIEW	Manage views	All
WITH name AS (...)	Define a CTE	All

CREATE INDEX / DROP INDEX	Manage indexes	All
EXPLAIN QUERY PLAN	Show query execution plan	SQLite
VACUUM	Reclaim disk space	SQLite
BEGIN TRANSACTION / COMMIT / ROLLBACK	Transaction control	All
BEGIN EXCLUSIVE TRANSACTION	Acquire exclusive lock	SQLite
CREATE TRIGGER	Define an automatic action	SQLite/MySQL
INSTEAD OF (trigger)	Override view operations	SQLite
CREATE PROCEDURE / CALL	Stored procedures	MySQL/PostgreSQL
GRANT ... TO user	Assign privileges	MySQL/PostgreSQL
PREPARE / EXECUTE	Prepared statements	All

CS50 Introduction to Databases — Complete Study Notes
Content based on CS50 course lectures (SQLite, PostgreSQL, MySQL)